Failure (a bad split, and a k comparison) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus why brute-force search, and "just raise k," both stop working at real scale). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

You're doing two things on purpose here: breaking a document badly, and then measuring what different search widths (`k`) actually buy you.

First, take a short, multi-paragraph test document, where one paragraph has a concrete fact — a sentence naming a specific date and place. Split the whole document into small, fixed-size chunks (say, 40 characters each), with no regard for where sentences or words end. Because the chunk size is small and doesn't know anything about sentence structure, the fact's sentence will very likely get sliced into two separate chunks.

Then embed every chunk, and write a `retrieve()` function that: embeds the question, compares it against every stored chunk with cosine similarity, and returns the closest `k` chunks.

Ask your fact question with `k=1` first. Notice the single chunk you get back usually only has *part* of the fact. Then run the exact same question again at `k=1` , `k=3` , and `k=10` , timing each call. Watch two separate things: how much more complete the fact gets as `k` grows, and how little the actual search time changes at this tiny scale.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Two functions carry this exercise: a chunker, and a retriever.

`chunk_by_chars(text: str, chunk_size: int) -> list[str]` is a deliberately "dumb" splitter — it cuts the string every `chunk_size` characters, no matter what's there.

`retrieve(question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int) -> list[tuple[float, str]]` embeds the question once, scores every stored `(chunk_text, embedding)` pair against it with cosine similarity, sorts by score descending, and returns the top `k`.

The exact pieces:

- A `while start < len(text):` loop slicing `text[start:start+chunk_size]`, moving `start` forward by `chunk_size` each time.
- `time.perf_counter()` — a monotonic clock, better than `time.time()` for measuring short durations.
- `list[tuple[str, list[float]]]` for storage — a plain Python list of `(chunk_text, embedding)` pairs is enough at this scale; no real vector database needed.

The timing you did above barely moves between `k=1`, `k=3`, and `k=10` — and that result is easy to over-generalize from. It only barely moves because `retrieve()` here compares your question against a *handful* of chunks, one by one, in a plain Python loop. That's brute-force search: cost grows in direct proportion to how many chunks exist, regardless of `k`. At a few dozen chunks, brute-force is effectively free. At a few hundred thousand — a real company's documentation, ticket history, or knowledge base — comparing against every single one for every single question stops being free, and a real vector database (Chroma, FAISS, a hosted vector store) uses an *approximate* nearest-neighbor index instead: a data structure built in advance that finds very-likely-closest chunks in roughly constant time, without touching every stored vector for every query. This document's Core Concepts describes this as the actual job of a "vector store" — this exercise's brute-force loop is a stand-in for it, not the real thing.

The second thing worth separating out: fixing the "half the fact went missing" bug you caused on purpose here is not the same problem as picking a good `k`. `k` decides how many results to return; where the boundary falls is decided by the chunker. Raising `k` from 1 to 10 hides the symptom for this one tiny document (where there's nowhere else for the missing half to hide) — it does not fix the cause, and won't reliably help at all once the corpus is bigger (see `chunk_boundary_split`'s solution for why). The actual fix is chunking with overlap, applied *before* any of this timing comparison happens.

The extra pieces:

- Measure `retrieve()`'s timing again, but against a much larger synthetic chunk list (repeat/pad your test document hundreds of times) — watch the per-query time actually start to grow with corpus size, unlike the near-flat numbers from a few dozen chunks. Embed those hundreds of chunks in **batches** (`input=` a list of up to 2,048 texts per call), not one call per chunk — the `get_embeddings_batch` idea from the Basic exercise.
- Re-run the whole exercise using `chunk_by_chars_with_overlap` (from `chunking_methods`, Intermediate section) instead of the plain `chunk_by_chars`, and confirm `k=1` alone now returns the complete fact. The rule: any text up to `overlap` characters long always fits whole inside some chunk — the fact sentence is 71 characters, so use `chunk_size=120, overlap=80`.

Difference between Basic and Intermediate: Basic observes that timing barely changes across `k` values, at a scale too small to show why that's misleading. Intermediate also explains *why* it barely changes (brute force over a handful of chunks), shows what happens once the corpus is actually large, and separates "the split-fact bug" (a chunking problem, fixed by overlap) from "how wide to search" (a `k`/cost trade-off) — two genuinely different decisions that are easy to mix up after watching `k` alone seem to fix things in a tiny example.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a test document: a few short paragraphs, one of which has a sentence
naming a specific date and place (your "fact")

split the whole document into small fixed-size chunks (like 40 characters),
ignoring word/sentence breaks
print the chunks, with their index numbers
    -> find the chunk (or two) where the fact's sentence got cut in half

get an embedding for every chunk
keep them as a list of (chunk_text, embedding) pairs

make a retrieve function that takes a question, the chunks+embeddings, and k:
    embed the question
    score every chunk against the question with cosine similarity
    sort by score, highest first
    return the top k

ask your fact question with k=1
    -> print the single chunk that comes back
    -> notice it's missing part of the fact

time retrieve() at k=1, k=3, and k=10:
    for each k: start a timer, call retrieve(), stop the timer
    print the chunks returned and how long it took, for each k

compare: does the completeness of the fact improve as k grows?
         does the search time change much?
```

Here are the core functions, almost complete — try running them and reading them line by line:

```
# chunking_practice.py — Failure section
import math
from openai import OpenAI

client = OpenAI()
```

```
def chunk_by_chars(text, chunk_size):
    chunks = []
    start = 0
    while start < len(text):
        chunk = text[start:start + chunk_size]
        chunks.append(chunk)
        start += chunk_size
    return chunks

def get_embedding(text):
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a, b):
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def retrieve(question, chunks_with_embeddings, k):
    question_embedding = get_embedding(question)
    scored = []
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]
```

What's missing: your `TEST_DOCUMENT`, building `chunks_with_embeddings`, and the script that prints chunks, runs `retrieve()` at `k=1` alone, then times it at `k=1`, `k=3`, and `k=10`. Add those yourself, then check the Solution.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
# chunking_practice.py — Failure section
import math
import time
from openai import OpenAI

client = OpenAI()

def chunk_by_chars(text: str, chunk_size: int) -> list[str]:
    chunks: list[str] = []
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        start += chunk_size
    return chunks

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def retrieve(
    question: str, chunks_with_embeddings: list[tuple[str, list[float]]], k: int
) -> list[tuple[float, str]]:
    question_embedding = get_embedding(question)
    scored: list[tuple[float, str]] = []
```

```
    for chunk_text, embedding in chunks_with_embeddings:
        score = cosine_similarity(question_embedding, embedding)
        scored.append((score, chunk_text))
    scored.sort(reverse=True)
    return scored[:k]
```

What's missing: `TEST_DOCUMENT`, a `main()` that builds `chunks_with_embeddings`, prints the chunks, runs `retrieve()` at `k=1` alone, then loops over `k` in `(1, 3, 10)` timing each call with `time.perf_counter()`. Write that yourself, then compare against the Solution.

Once that's working, add the missing variable — corpus size — and the real chunking fix:

```
build a much bigger chunk list: your TEST_DOCUMENT repeated/padded until
    there are hundreds (or low thousands) of chunks
embed them in slices of 1,000 per embeddings call, not one call each

time retrieve() against this bigger list at the same k values (1, 3, 10)
    -> compare against the timings from the small chunk list

separately: build chunks_with_overlap using chunk_by_chars_with_overlap
    instead of chunk_by_chars, on the ORIGINAL small TEST_DOCUMENT,
    with chunk_size=120, overlap=80 (overlap >= the 71-char fact)

run retrieve() at k=1 against the overlapping chunks
    -> does the single top chunk now contain the whole fact?
```

Here's almost the whole thing — fill in the missing piece yourself:

```
# chunking_practice.py — Failure section
def chunk_by_chars_with_overlap(
    text: str, chunk_size: int, overlap: int,
) -> list[str]:
    chunks: list[str] = []
    step = chunk_size - overlap
    start = 0
    while start < len(text):
        chunks.append(text[start:start + chunk_size])
        # your turn: advance start by step, not chunk_size
        ...
    return chunks


def build_large_corpus(document: str, repeats: int) -> str:
    # your turn: return `document` repeated `repeats` times, so you have a
```

```
    # much bigger (but still synthetic) document to chunk and time against
    ...
```

Fill in both functions, run the timing comparison at small vs. large corpus size, and run the overlap comparison at `k=1`, then compare all of your finished versions against the Solution.

Difference between Basic and Intermediate: Basic times `retrieve()` at 3 values of `k`, against a handful of chunks, and sees almost no difference. Intermediate adds the missing variable — corpus size — and shows the timing difference that was hiding the whole time, plus applies the real chunking fix (overlap) instead of just widening `k` to paper over the boundary split.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution